

SIH26011: 3D ULPIN & Vertical Property Mapping System

Complete Developer & Designer Handover Specification

1. Executive Summary & Product Vision

- **Project Name:** Bhu-Aadhaar 3D (3D-VLAD: Vertical Land Administration Domain)
- **Problem Statement ID:** SIH26011
- **Ministry:** Ministry of Rural Development / Department of Land Resources (DoLR)
- **Core Mission:** Bridge India's 2D land registry system into an interactive, volumetric 3D digital cadastre for high-rise buildings, apartment units, basements, and parking spaces, eliminating multi-mortgage fraud and boundary disputes.

2. Comprehensive Application Flows & User Journeys

User Journey 1: Citizen / Homebuyer (Public Verification Flow)

1. **Landing Page:** Citizen lands on a clean, modern GovTech portal with a prominent search bar: "Search by 2D ULPIN, Survey Number, or City/Building Name".
2. **Search & Auto-Complete:** Citizen enters a building name (e.g., "Galaxy Heights, Nashik") or land survey number.

3. **Map Centering:** The 2D map glides smoothly and centers on the target land parcel.
4. **3D Extrusion:** The flat building footprint extrudes upwards into an interactive 3D model with realistic floor heights.
5. **Interactive Unit Selection:** Citizen clicks on Floor 4 → Unit 402. The camera smoothly orbits and zooms toward Floor 4.
6. **Property Card Sheet:** A right-hand panel slides open showing:
 - Legal 3D-ULPIN (14MH2704291845-FL04-U402-Z12.5)
 - Ownership status (Registered Owner Name, Registration Date)
 - Carpet area and balcony measurements
 - Assigned Basement Parking Slot (e.g., P-14)
 - Loan/Mortgage Status (e.g., "Active Mortgage with SBI" or "Clean Title - No Liens")
7. **Download Digital 3D Property Card:** Citizen can click "Download Verified 3D Property Card" (PDF with QR code).

User Journey 2: Revenue Officer / Municipal Admin (Mutation & Approval Flow)

1. **Authentication:** Officer logs in via secure role-based access.
2. **Pending Mutation Requests:** Officer views a list of pending flat transfers, builder subdivision plans, or new unit registrations.
3. **3D Spatial Audit:** Clicking a request opens the 3D model. The proposed flat or newly added floor is highlighted in amber.
4. **Conflict Check:** Officer runs "Automated Conflict Scan". The system checks whether the builder's declared boundary overlaps with common areas, fire exits, or adjacent units.
5. **One-Click Approval:** If conflict score is 0, the officer clicks "Approve Mutation". The system assigns the official permanent 3D ULPIN and updates the immutable ledger.

User Journey 3: Bank / Financial Institution (Collateral Verification Flow)

1. **Lien Check Interface:** Bank loan officer enters the 3D ULPIN provided in a loan application.

2. **Instant Verification:** The system displays the exact 3D location of the flat and checks if any other financial institution has already registered a lien.
3. **Double-Mortgage Prevention:** If an active lien exists, the system flags a high-priority warning: *"Active Mortgage Registered by Bank of Baroda on 14-Jan-2025"*, completely preventing multi-bank collateral fraud.

3. UI/UX Design Specifications (For Designers)

Design Philosophy

- **Aesthetic:** "Future GovTech" — Clean, modern, trustworthy, high-tech GIS interface inspired by modern aerospace and digital twin dashboards (like Linear, Vercel, and Cesium).
- **Theme:** Default to Dark Mode for high-contrast 3D WebGL viewing, with an accessible Light Mode toggle for official daytime use.

Color Tokens & Palette

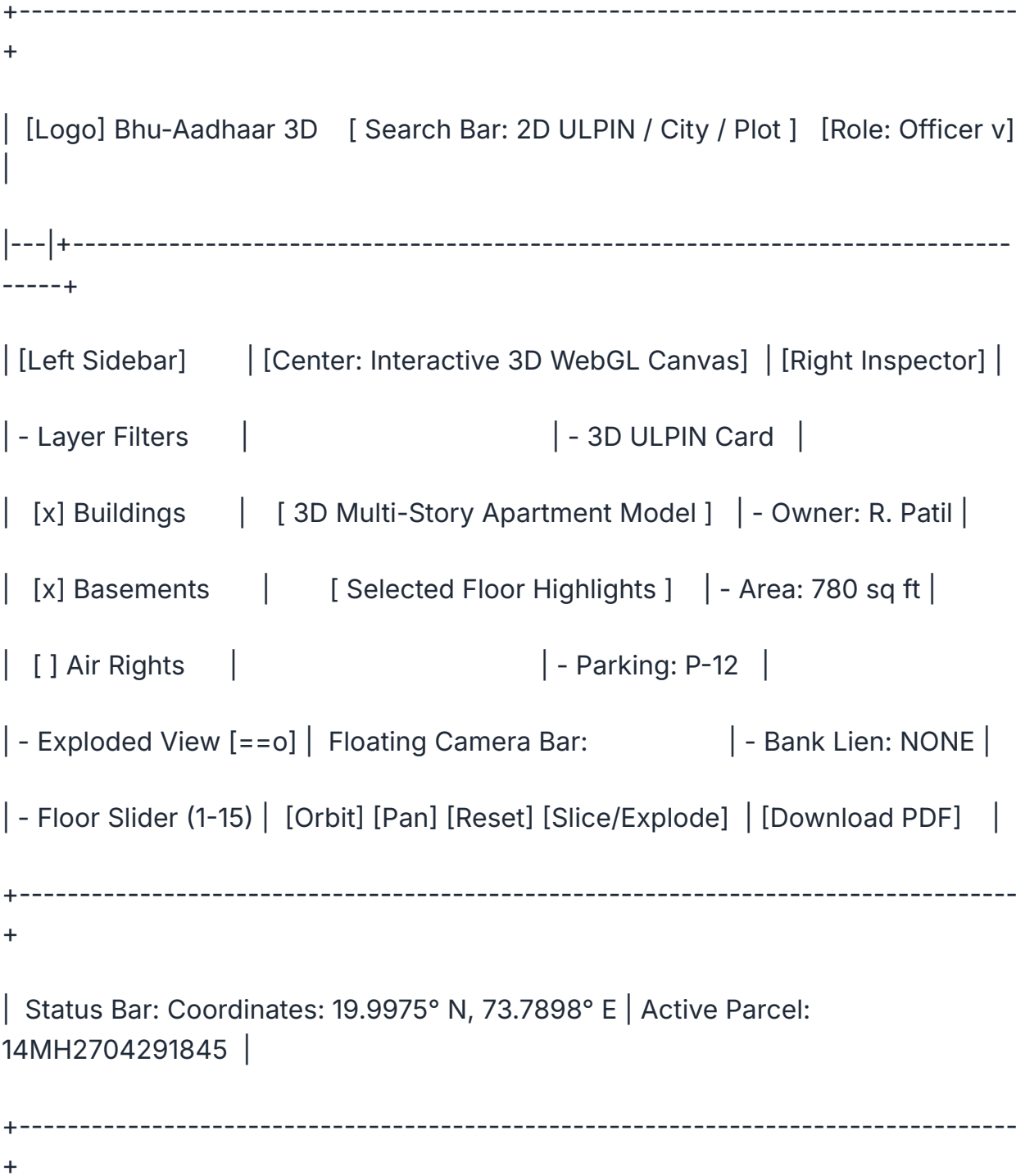
- **Background Deep (Canvas/Map Base):** #0B0F19 (Rich Navy-Black)
- **Surface/Card Container:** #161F30 (Translucent glassmorphism with 12px blur, border: 1px solid rgba(255, 255, 255, 0.08))
- **Primary Brand / Accent:** #2563EB (India Digital Blue) / Hover: #3B82F6
- **Success / Verified State:** #10B981 (Emerald Green - 3D selected unit glow)
- **Conflict / Fraud Alert State:** #EF4444 (Ruby Red - pulsing wireframe for disputes)
- **Pending / In-Review State:** #F59E0B (Amber Orange)
- **Text Primary:** #F9FAFB (High contrast white)
- **Text Muted:** #9CA3AF (Secondary labels and coordinate values)

Typography

- **Font Family:** Inter or Plus Jakarta Sans for clean, readable GovTech interfaces.

- **Code / ULPIN Display Font:** JetBrains Mono or Fira Code (monospace for unambiguous alphanumeric ID verification).

Viewport Layout & Wireframe Blueprint



Micro-Interactions & Animations

1. **Raycast Hover:** When the mouse hovers over an apartment unit, its opacity changes from 0.4 to 0.7 with a subtle cyan edge wireframe.
2. **Unit Selection Glow:** Clicking an apartment unit transitions its mesh material into emerald green (`#10B981`) with a soft bloom glow, while non-selected units dim to 0.2 opacity.
3. **Exploded View Toggle:** Clicking "Explode Floors" animates every floor moving vertically apart along the Z -axis by 2.5 meters (using a smooth cubic easing curve), allowing users to see inside middle floors effortlessly.
4. **Conflict Shake Animation:** If a user tests duplicate parking or unauthorized terrace construction, the affected 3D geometry pulses red with a gentle 3-frame haptic shake.

4. Technical Architecture & Developer Instructions

System Architecture Overview

- **Client Layer:** Pure HTML5, Modern CSS3 (Glassmorphism, Flexbox/Grid), and Vanilla JavaScript (ES6 Modules). No Node.js build steps or npm dependencies required.
- **3D Graphics Engine:** Three.js loaded via CDN (cdnjs / jsdelivr) along with OrbitControls.
- **Database & Spatial Backend:** Supabase (PostgreSQL + PostGIS) connected directly via the official Supabase JavaScript client CDN script (@supabase/supabase-js).
- **Optional Backend:** Node.js (Express) or Python (FastAPI) on Render free tier, or direct Supabase client-side queries with Row-Level Security.
- **Deployment:** GitHub Pages or Vercel / Netlify (zero-config, instant 1-click deployment).

Vanilla HTML/CSS/JS File Structure

- `index.html`: Canvas container, top navigation bar, search bar, and slide-out property inspector card.

- style.css: Modern GovTech dark theme, glassmorphism cards, responsive viewport styling.
- scene3d.js: Three.js scene setup, procedural floor and unit box extrusion, camera controls, raycasting mouse click detection, and exploded view animation.
- app.js: 3D ULPIN generation function, Supabase database queries, UI event listeners, and duplicate-allocation alert logic.

Database Schema (Supabase PostgreSQL / PostGIS)

-- 1. Base 2D Land Parcel Table

```
CREATE TABLE parcels (

  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  ulpin_2d VARCHAR(14) UNIQUE NOT NULL,

  survey_number VARCHAR(50) NOT NULL,

  state VARCHAR(50) DEFAULT 'Maharashtra',

  district VARCHAR(50) NOT NULL,

  taluka VARCHAR(50) NOT NULL,

  village VARCHAR(50) NOT NULL,

  boundary_geojson JSONB NOT NULL,

  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()

);
```

-- 2. Building Structure Table

```
CREATE TABLE buildings (
```

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

parcel_id UUID REFERENCES parcels(id) ON DELETE CASCADE,

building_name VARCHAR(100) NOT NULL,

total_floors INTEGER NOT NULL,

basement_floors INTEGER DEFAULT 1,

height_meters NUMERIC(6, 2) NOT NULL,

footprint_geojson JSONB NOT NULL

);
```

-- 3. Individual 3D Units (Flats / Commercial Units)

```
CREATE TABLE units (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

building_id UUID REFERENCES buildings(id) ON DELETE CASCADE,

ulpin_3d VARCHAR(50) UNIQUE NOT NULL,

unit_number VARCHAR(20) NOT NULL,

floor_number INTEGER NOT NULL,

elevation_bottom_z NUMERIC(6, 2) NOT NULL,

elevation_top_z NUMERIC(6, 2) NOT NULL,

carpet_area_sqft NUMERIC(8, 2) NOT NULL,
```

```
    bounding_box_3d JSONB NOT NULL, -- Coordinates: [minX, minY, minZ, maxX,
maxY, maxZ]
```

```
    owner_name VARCHAR(100) NOT NULL,
```

```
    owner_aadhaar_hash VARCHAR(64),
```

```
    property_tax_id VARCHAR(50),
```

```
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
```

```
);
```

```
-- 4. Parking Allocations (Basement / Stilt)
```

```
CREATE TABLE parking_slots (
```

```
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
    building_id UUID REFERENCES buildings(id) ON DELETE CASCADE,
```

```
    slot_number VARCHAR(20) NOT NULL,
```

```
    floor_level VARCHAR(10) NOT NULL, -- e.g., 'B1', 'B2', 'Stilt'
```

```
    assigned_unit_id UUID REFERENCES units(id) ON DELETE SET NULL,
```

```
    slot_coordinates_3d JSONB NOT NULL,
```

```
    CONSTRAINT unique_slot_per_building UNIQUE (building_id, slot_number)
```

```
);
```

```
-- 5. Encumbrance / Bank Mortgage Registry
```

```
CREATE TABLE encumbrances (
```

```

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

unit_id UUID REFERENCES units(id) ON DELETE CASCADE,

bank_name VARCHAR(100) NOT NULL,

loan_account_number VARCHAR(50) NOT NULL,

loan_amount NUMERIC(12, 2) NOT NULL,

status VARCHAR(20) DEFAULT 'ACTIVE', -- 'ACTIVE' or 'DISCHARGED'

registration_date DATE NOT NULL

);

```

Algorithmic 3D ULPIN Generation Function

```

<!-- index.html CDN Setup --><script
src="https://cdnjs.cloudflare.com/ajax/libs/three.js/0.160.0/three.min.js"></s
cript><script
src="https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2"></script>

export function generate3DUlpin(params) {
  const floorFormatted = params.floorNumber < 0
    ? `B${Math.abs(params.floorNumber)}`
    : `FL${params.floorNumber.toString().padStart(2, '0')}`;
  const unitClean = params.unitNumber.toUpperCase().replace(/\s+/g, '');
  const zHeight =
`${params.elevationBottomZ.toFixed(1)}-${params.elevationTopZ.toFixed(1)}M`;
  return `${params.parcelUlpin2d}-${floorFormatted}-U${unitClean}-${zHeight}`;
}

const unitClean = params.unitNumber.toUpperCase().replace(/\s+/g, '');
const zHeight =
`${params.elevationBottomZ.toFixed(1)}-${params.elevationTopZ.toFixed(1)}M;

// Example Output: "14MH2704291845-FL04-U402-12.5-15.5M"

```

Vanilla JavaScript Three.js Raycasting

JavaScript

```
const raycaster = new THREE.Raycaster();
const mouse = new THREE.Vector2();
window.addEventListener('click', (event) => {
  mouse.x = (event.clientX / window.innerWidth) * 2 - 1;
  mouse.y = -(event.clientY / window.innerHeight) * 2 + 1;
  raycaster.setFromCamera(mouse, camera);
  const intersects =
    raycaster.intersectObjects(scene.children, true);
  const unitHit = intersects.find(hit =>
    hit.object.userData?.isUnit);
  if (unitHit) {
    const selectedMesh = unitHit.object;
    selectedMesh.material.color.setHex(0x10b981);
    updateUI(selectedMesh.userData);
  }
});
```

Supabase & Conflict Logic (Vanilla JS)

JavaScript

```
const { createClient } = supabase;
const client = createClient('URL', 'KEY');
async function checkConflict(buildingId, slotNumber) {
  const { data, error } = await client.from('parking_slots')
    .select('*', units(unit_number, owner_name))
    .match({ building_id: buildingId, slot_number: slotNumber }).single();
  if (data && data.assigned_unit_id) {
```

```
    showConflictAlert(data.units.unit_number);  
  }  
}
```

5. Team Division of Labor (36-Hour Hackathon Sprint)

Role	Responsibility
Developer 1	3D Graphics Lead: Builds the 3D scene using Vanilla Three.js, procedural box extrusions, and exploded view animations via CDN scripts.
Developer 2	Data & API Lead: Manages Supabase PostgreSQL/PostGIS, writes 3D ULPIN generator, and implements conflict logic using direct CDN client queries.
Developer 3	Frontend & UI Lead: Crafts index.html and style.css, building the search bar, sidebar filters, and property inspector without external frameworks.
Designer / UI Lead	Design & Pitch Lead: Prepares Figma component library, SVG icon set, color tokens, and crafts the 3-minute pitch slide deck and live demonstration script.

6. Definition of Done (Hackathon Ready Checklist)

- ☐ 3D canvas loads smoothly at 60 FPS in Google Chrome without errors.
- ☐ Clicking units triggers the Vanilla JS raycaster to display legal 3D ULPINs via the app.js logic.
- ☐ Raycasting correctly reads floor number and \$Z\$-axis elevation in meters.
- ☐ Attempting duplicate parking assignment triggers the red conflict alert.
- ☐ Application is deployed live on GitHub Pages or Vercel using zero-npm instant deployment.
- ☐ Final review by on .